Réalisation d'un bras artificiel

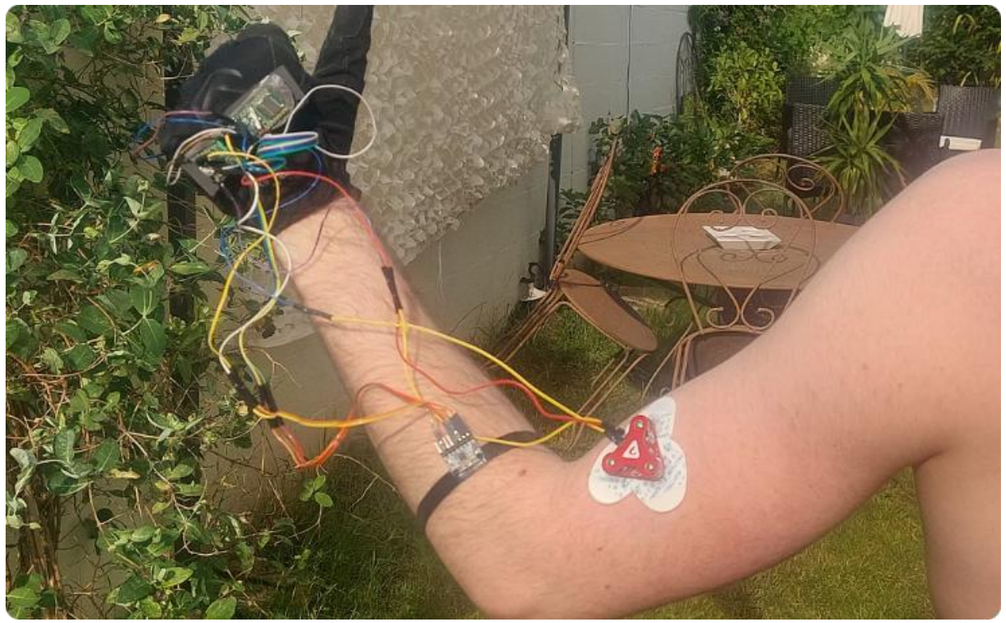
Transition, transformation, conversion

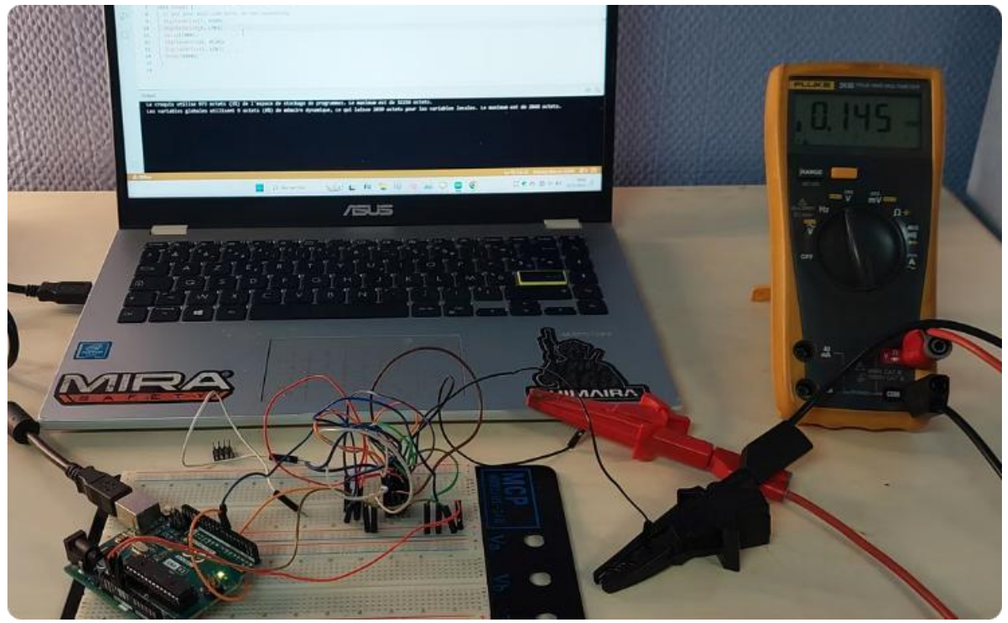
Session 2025

No élève 53245 – Arthur Ripoll



Préambule



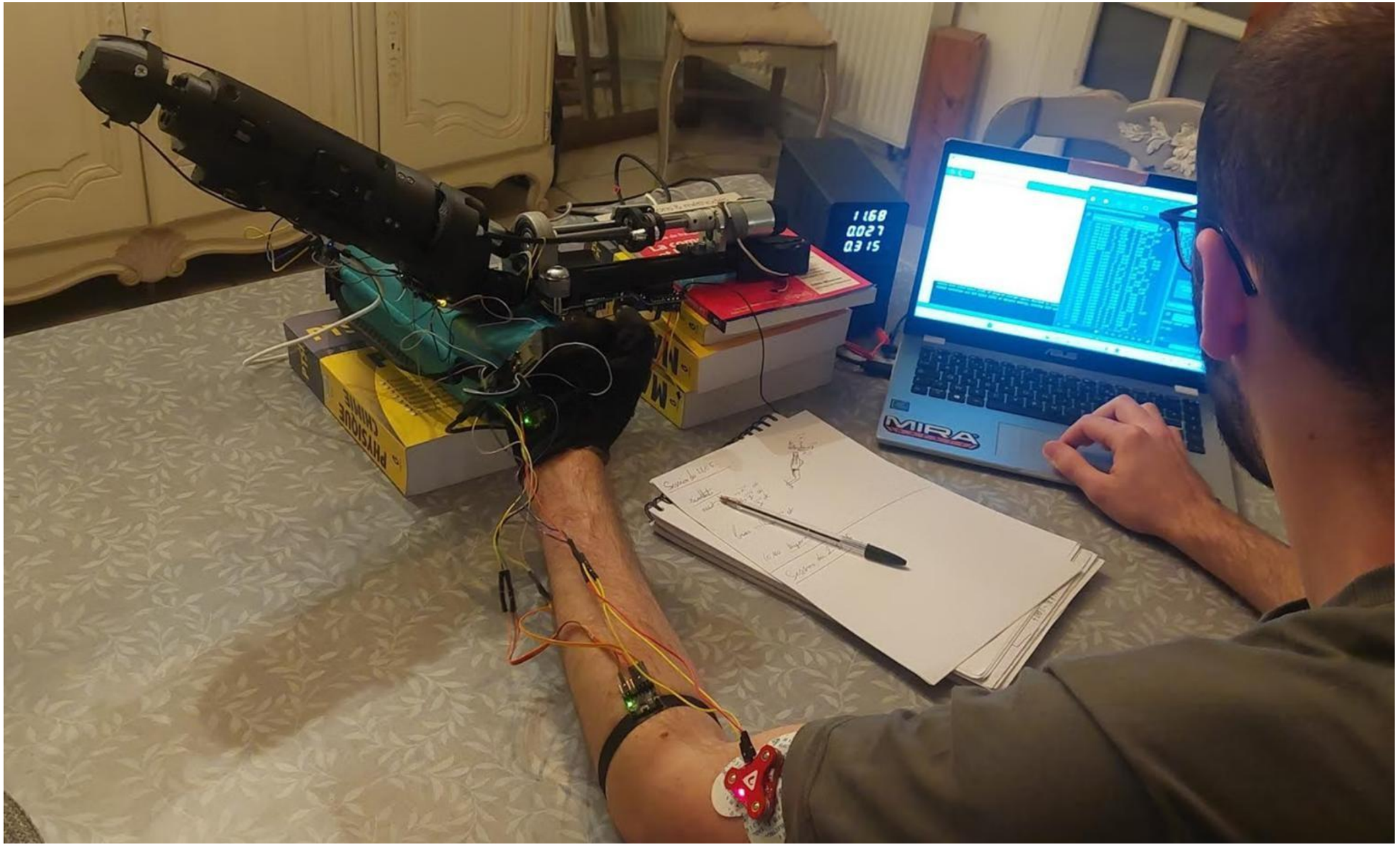


Le Champ de la Biomécanique

La biomécanique englobe les dispositifs d'imagerie médicale et les technologies de remplacement d'organes ou de membres défaillants. Nous nous concentrons ici sur les implants médicaux externes.

Répondre aux Besoins de l'Utilisateur

Il est essentiel de s'aligner précisément sur les attentes et les besoins individuels de chaque utilisateur pour assurer une association durable et efficace.



(vérification du bon fonctionnement du roulis de l'avant-bras)

Objectifs du TIPE : Conception et Fonctionnalité

L'objectif de ce projet est de déterminer dans quelle mesure nos connaissances de classe préparatoire permettent de concevoir un bras artificiel capable de réaliser les fonctions d'un bras humain.

Concevoir un bras artificiel

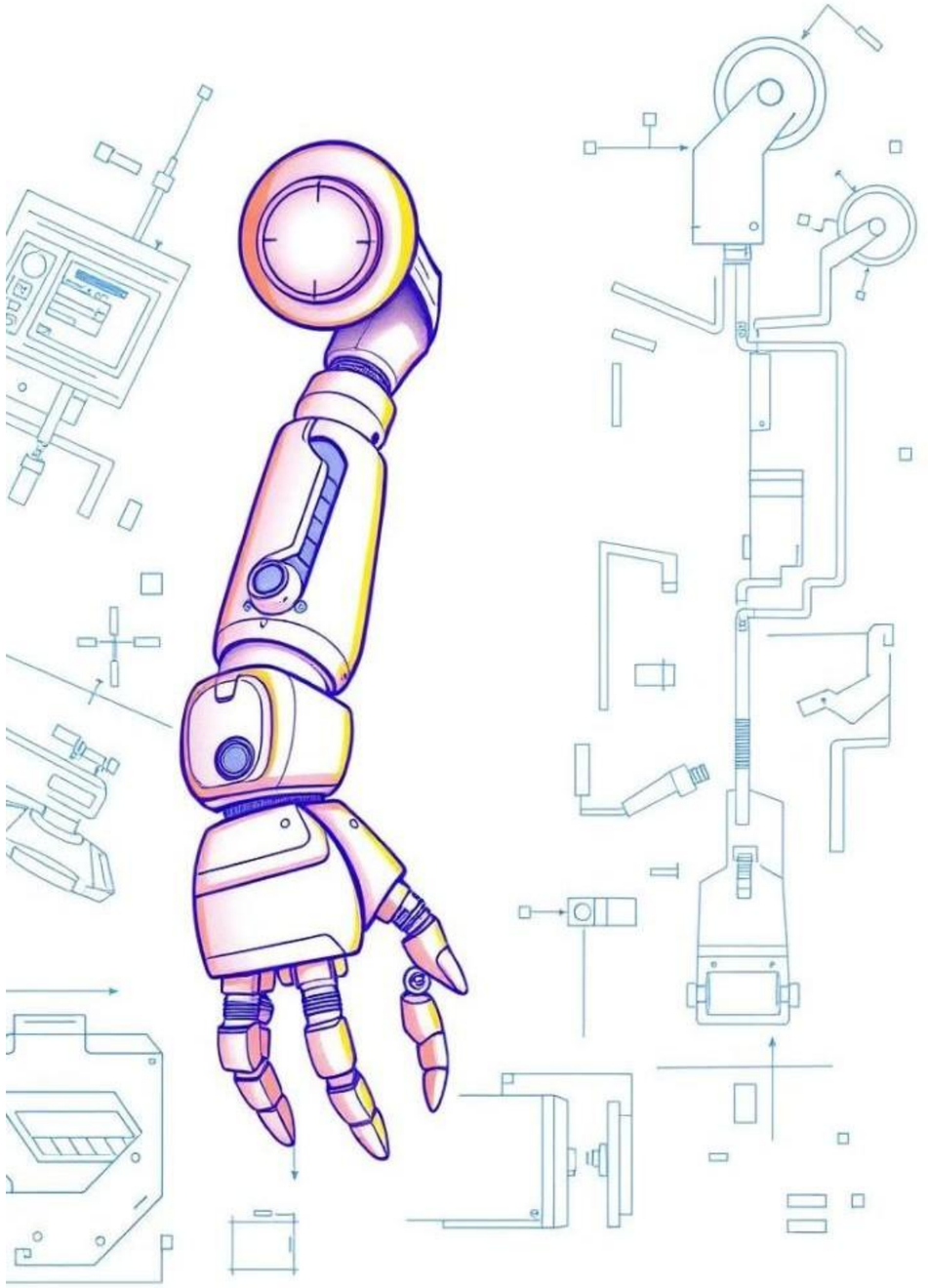
Développer un design fonctionnel et réaliste pour la prothèse.

Récupérer les informations de position

Acquérir toutes les données nécessaires sur la position du bras de l'utilisateur.

Établir une communication entre les différents composants

Assurer une la réactivité de la prothèse vis-à-vis de la consigne provenant de l'utilisateur.



Sommaire :

1

Capteurs

Assurer la conversion des signaux physiques en données numériques exploitables

2

Asservissement

Concevoir des solutions d’asservissement pour le pilotage dynamique et précis des équipements

3

Transmission de puissance

Concevoir et maîtriser les systèmes de transmission de puissance adaptés à notre système

4

Réseau

concevoir l’architecture des réseaux pour garantir la performance, la sécurité et la scalabilité des systèmes.

5

Programmation

Coder des programme respectant les exigences assurant la fiabilité des échanges réseau.

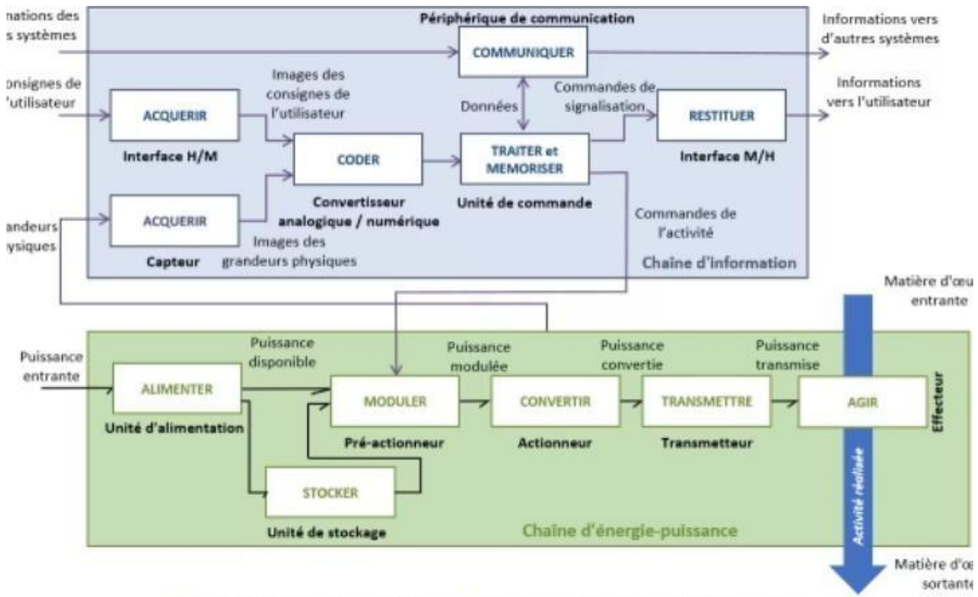


Figure 2 : Schéma spécifique de la chaine fonctionnelle d’un système

→ 1 : Acquisition et Traitement du Signal EMG :

Contraction Musculaire

Acquérir un signal résultant de la contraction d'un muscle pour le contrôle.

Capteur EMG

Utiliser l'électromyogramme (EMG) pour mesurer la contraction musculaire.

Traitement du Signal

Traiter le signal EMG, car il n'est pas directement exploitable en sortie de capteur.



Le capteur EMG présente des défis, car sa stabilité peut être affectée par plusieurs facteurs. Le traitement du signal est donc crucial pour rendre les données exploitables et assurer un contrôle précis de la prothèse.

Construction d'un filtre :

D'après la forme canonique d'un filtre du premier ordre dans le domaine de Laplace : →

$$H(p) = \frac{S(p)}{E(p)} = \frac{K}{\tau p + 1}$$

$$S(p) * (\tau p + 1) = k * E(p)$$

$$\Rightarrow \tau * \frac{ds}{dt}(t) + s(t) = k * e(t)$$

Cela nous donne donc la fonction suivante (en python) :

```python
def filtre1 (tau:float ,temps:list ,E:list ) -> list:
    S=[E[0]]
    for i in range (1,len(E)):
        pas = temps[i] - temps[i-1]
        s = pas/(tau+pas)*E[i]+tau/(tau+pas)*S[i-1]
        S.append(s)
    return(S)
```

→ *1 : Acquisition et Traitement du Signal EMG*

Prise de mesures de la position du poignet :

Choix : capteur 6-axes (MPU6050)

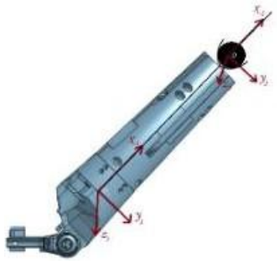


Ce capteur fait à la fois office d'accéléromètre et de gyroscope.

$X=aysin(c)sin\alpha+azsin(c)cos\alpha+axcos(c)cos\beta-aycos(c)cos\alpha sin\beta+azcos(c)sin\alpha sin\beta$

$Y=ax\cdot sin(\beta)+(ay\cdot sin(\alpha)+az\cdot cos(\alpha))\cdot cos(\beta)$

$Z=cos(c)(-axcos\beta+aysin\alpha+aycos\alpha sin\beta+azcos\alpha-azsin\alpha sin\beta)$



→ Ce sont les expressions des vecteurs de la base locale de la main (Vect(ax,ay,az) projetés dans le repère de l'avant-bras (X, Y, Z).

→ 1 : Acquisition et Traitement du Signal EMG

Détermination de la position de la main dans l'espace :

À l'aide de la formule de Varignon et grâce aux changements de bases effectués précédemment, on peut étudier la position de la main dans le repère local de l'avant-bras en effectuant une différence entre l'intégrale des torseurs cinématiques déplacée en le centre de la liaison sphérique. (voir ligne 312 à 363 du programme intitulé "carte réceptrice")

→ 2 : Asservissement :

1- Acquisition

Récupération des données provenant des capteurs



2- Mise en Forme du Signal

Traiter et mettre en forme les signaux EMG avant de les envoyer aux actionneurs.

3- Communication

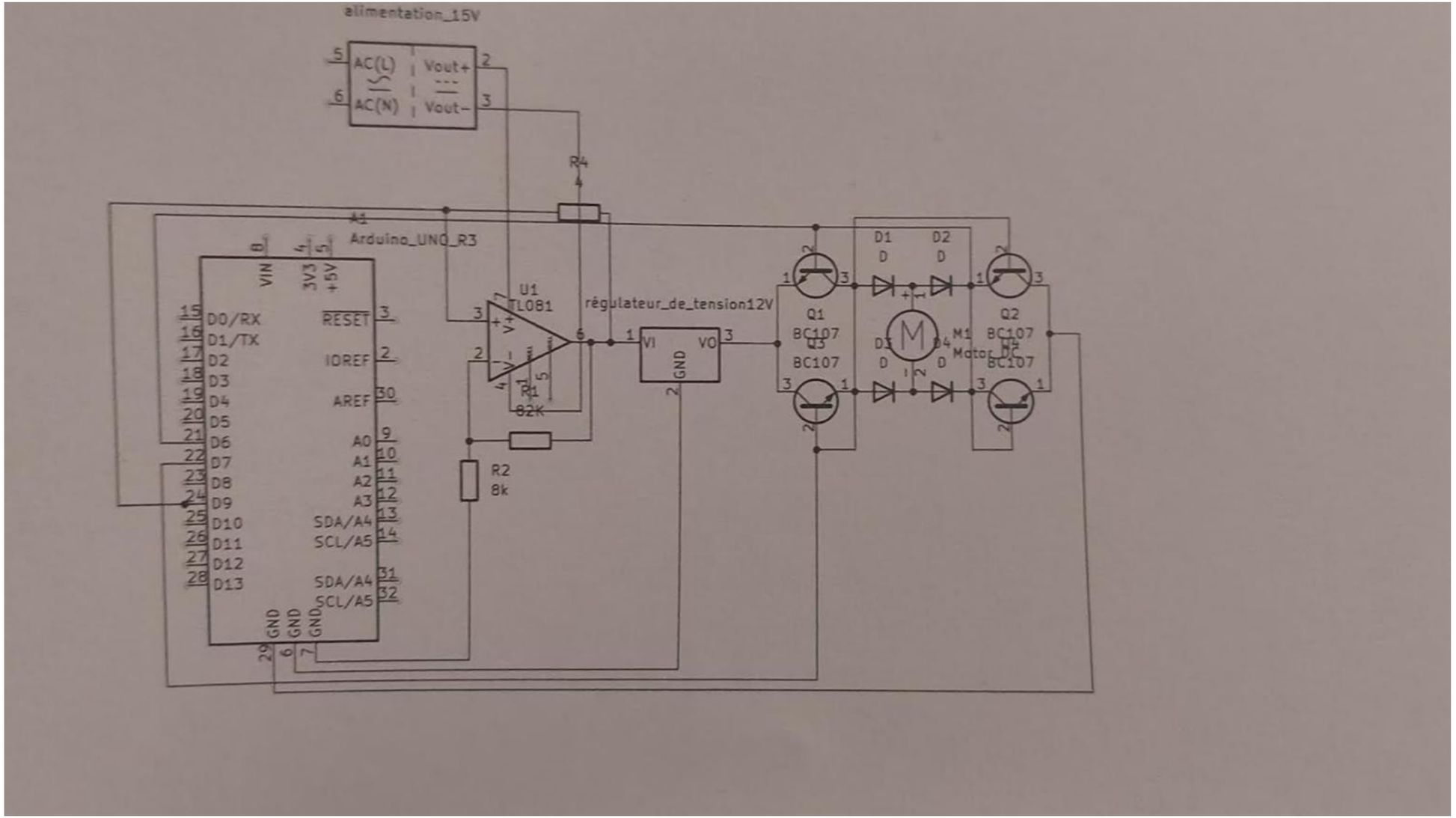
Établir une communication fluide entre le bras de l'utilisateur et le bras artificiel.

4- Asservissement

Asservir les actionneurs du bras artificiel pour une précision conforme aux exigences.

→ *2 : Asservissement*

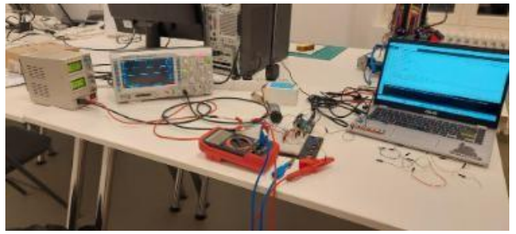
Conception d'un pont en H :



→ 2 : *Asservissement*

Causes de l'échec :

1. L'ALI délivrait un signal mais pas assez de courant (il fut remplacé par un transistor qui permettait de distribuer la puissance provenant de l'alimentation 15v)
2. Le transistor qui remplaçait l'ALI combiné à ceux qui composent le pont en H avaient une résistance interne trop élevée pour le signal provenant de l'Arduino. Cela erronait le signal PWM (Pulse-width modulation) provenant de la carte arduino uno r3.





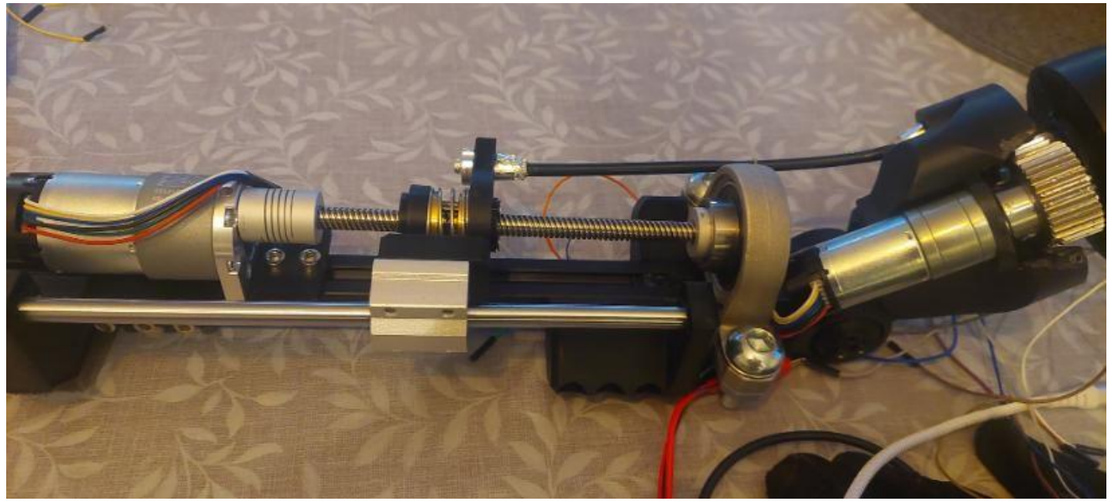


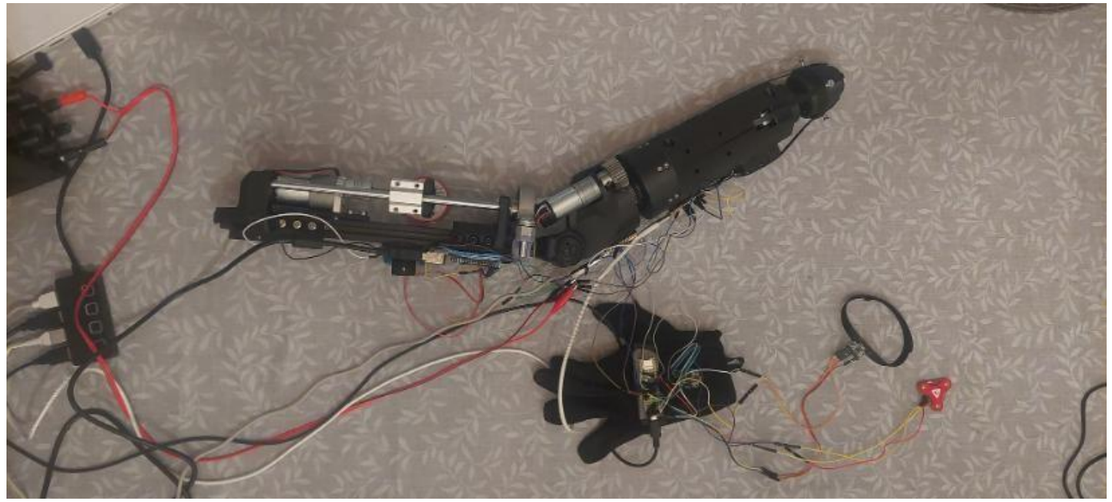
→ Solution :

L'achat d'un pont en H (L298N) fut une obligation afin de ne pas perdre plus de temps.

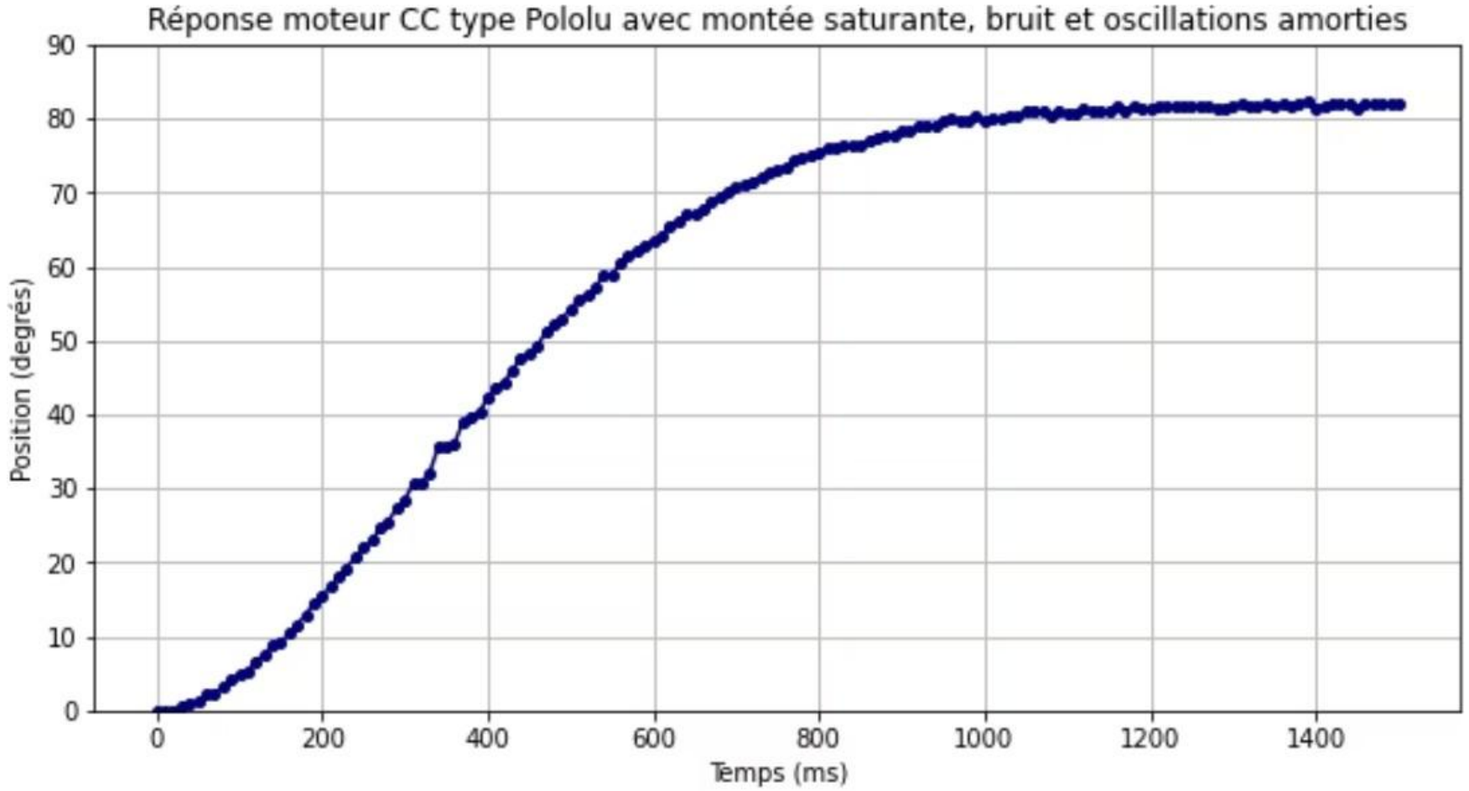
Asservissement du moteur à courant continu :

(voir le programme intitulé "asservissement moteur CC")





→ 2 : Asservissement

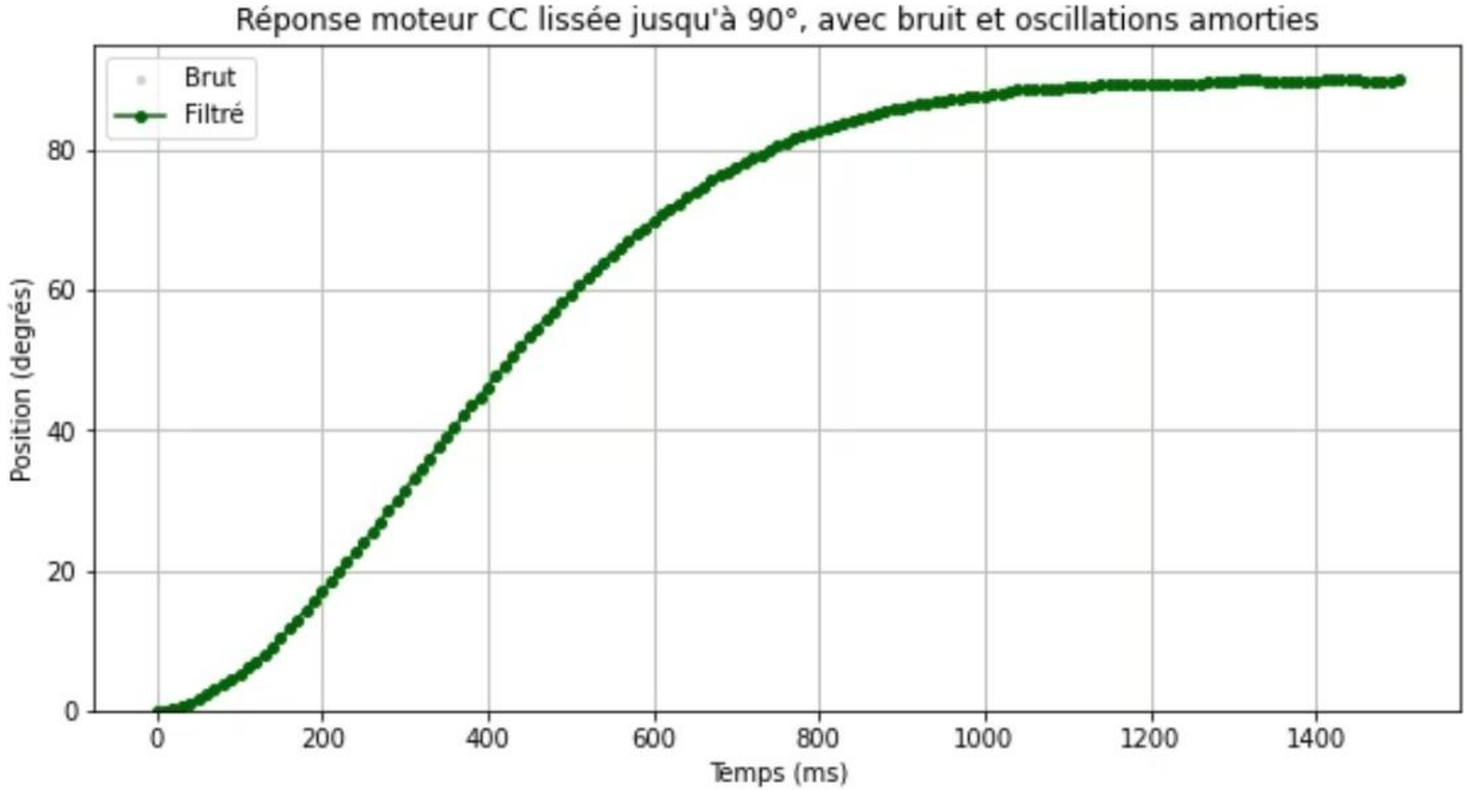


on constate :

- une erreur statique <8°
- la courbe est bruitée

Voici la courbe de sortie du moteur à courant continu quand on lui envoi une consigne en échelon de 90° (étant soumis à une tension de 9V).

→ 2 : Asservissement



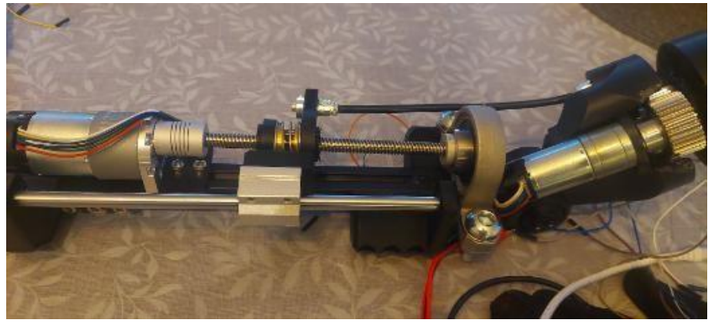
on constate :

- une erreur statique de 1°
- la courbe est raisonnablement lisse

Voici la courbe de sortie du moteur à la suite de notre programme le programme "asservissement moteur CC"

→ 3 : Transmission de puissance :

Cette phase finale intègre tous les composants et systèmes développés, garantissant que le bras artificiel est capable de réaliser les fonctions attendues. L'objectif est de démontrer la faisabilité de la conception et de son implantation potentielle.



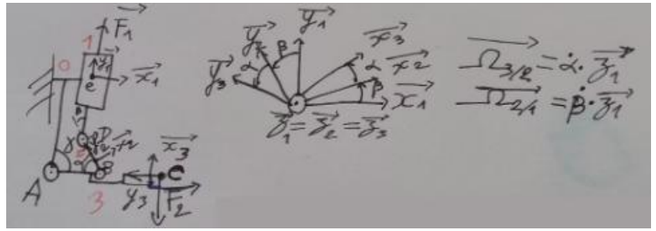






Calculs de statique et de cinématique :

1Dimensionnement du moteur



(avec F2 = 10kg, DB = 164mm, BA=55mm, AC=358mm)

$$\lambda = \cos(\gamma) - BD * \sin(\beta)$$

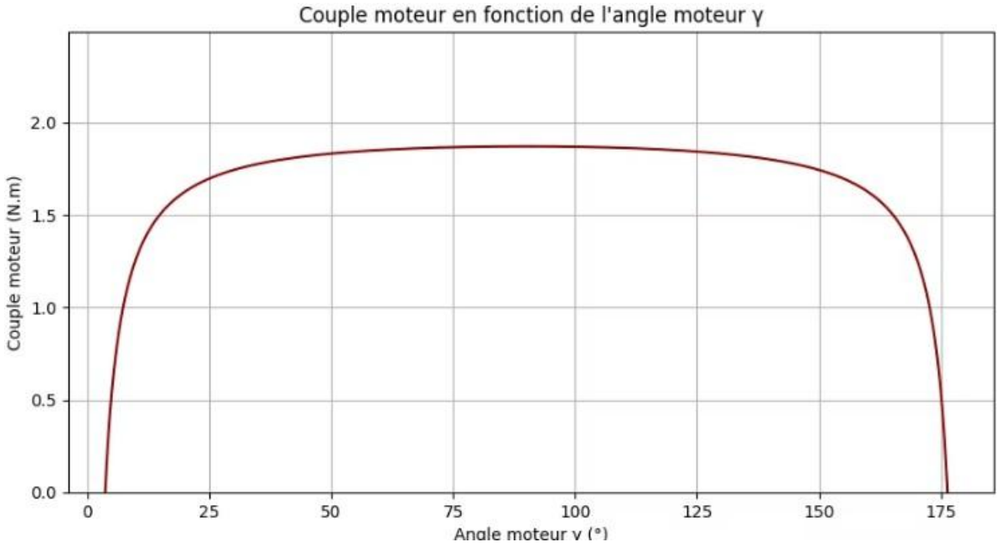
Principe fondamental de la statique : $(BA * \sin(\alpha) - DB * \sin(\beta)) * \overrightarrow{F1} - AC * \overrightarrow{F2}) * \overrightarrow{Z1} = 0$

$$\Rightarrow F1 = \frac{-(AC*F2)}{BA*\sin(\gamma)+DB*\sin(\beta)}$$

$$Cmot = \pm F1\left(\frac{pas}{2\pi}\right)$$

La gaine incompressible correspondant au tendon du biceps (reliant l'avant bras et le vérin) est représentée per la tige indéformable entre les deux liaisons pivots de centre (B et D).

→ 3 : Transmission de puissance



En faisant varier L'angle du coude (en A sur le schéma) de 0 à 180°, on constate que le couple moteur maximum occure quand l'angle du coude est égal à 90°. Et donc Cmot_max = 1.87

Environ 2Nm avec une marge de 6%.

Voici les spécifications techniques de notre

Motoréducteur + encodeur 37Y3530-43.8 (Pololu)

Nominal	1.5	0.147 N·m
Au blocage	18	1.765 N·m

Ce n'est donc pas suffisant pour qu'un patient puisse porter un pack de 6 bouteilles d'eau (**9.320 kg environ**).

[Mais cela convient pour notre maquette]

→ 4 : Construction du réseau :

Afin de ne pas surcharger les cartes électroniques, nous sommes contraints de répartir les tâches entre les différentes cartes.

Nous sommes contraints de bâtir un réseau.

→ Réseau en étoile :

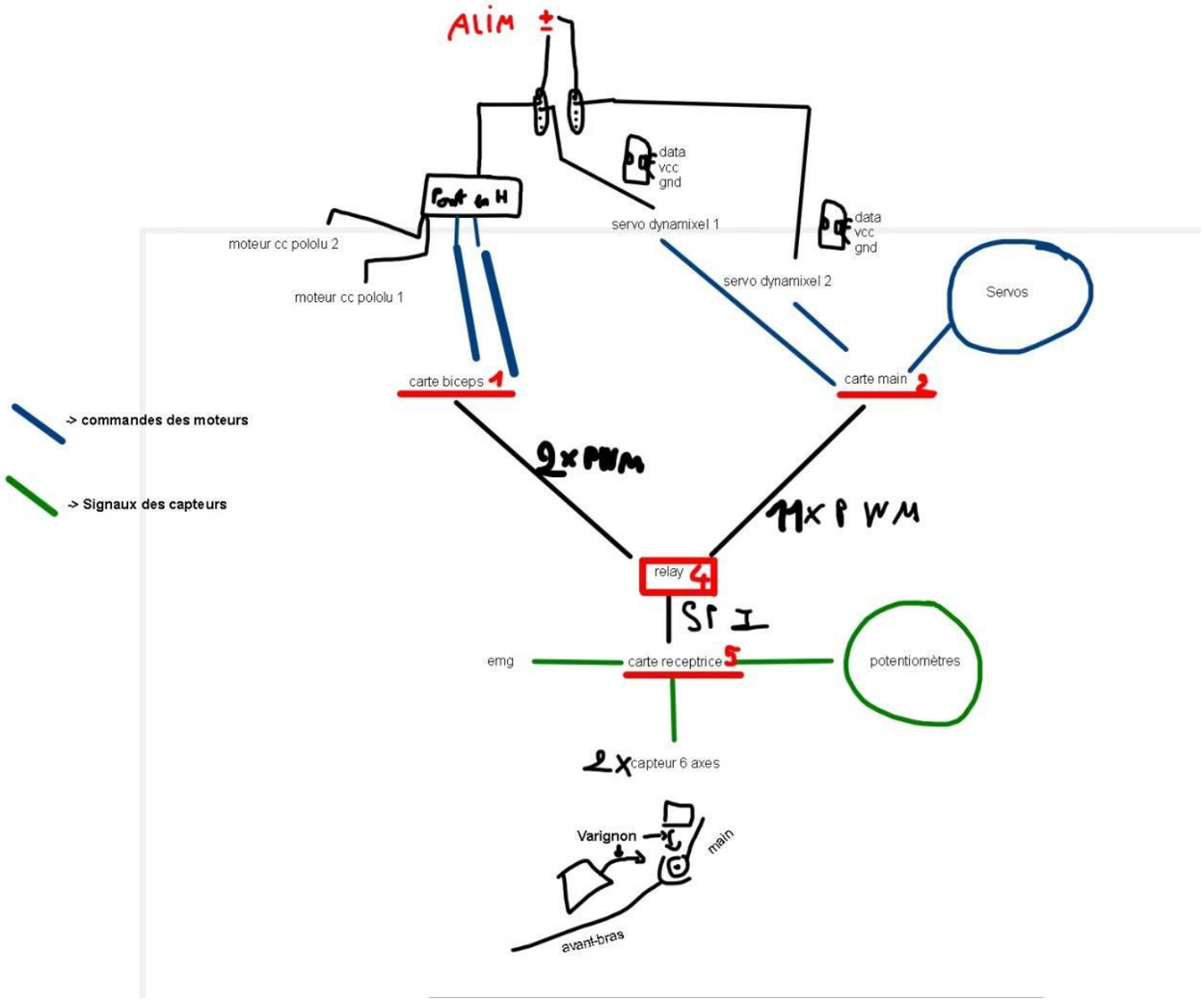
→ **Structure** : Tous les équipements connectés à un point central (commutateur ou hub)

+ Performance et latence : Faible latence : toutes les communications passent par le centre

+ Complexité de déploiement : Simple à déployer et gérer

- Câblage et interfaces : Plus complexe et demande plus de soudure ; chaque nœud a une liaison directe vers le centre

Structure du réseau :



(Voir annexes pour l'exploitation des différents types de transmissions de données)

Conclusion :

- Maquette se rapprochant des standards attendus.

Ces exigences seront atteignables avec des composants utilisés par des professionnels

- Malgré certaines embuches, le **projet fut complété à 100 %**

(Nous avons pas abordé le sujet des servos moteurs car cela ne s'inscrit pas dans le programme des deux années de PTSI-PT)

Cependant, certaines pistes afin de continuer le projets

ont déjà commencées à être explorées

Pistes en cours :

- Conception de la main

Pistes inexplorées pour le moment :

- Utilisation de l'intelligence artificielle afin de prédire les mouvements de l'utilisateur pour améliorer la réactivité de la prothèse au quotidien.
- Utilisation de capteurs piézo-électriques afin de permettre au patient de pouvoir tenir des objets sans les endommager

Réalisation de la main :

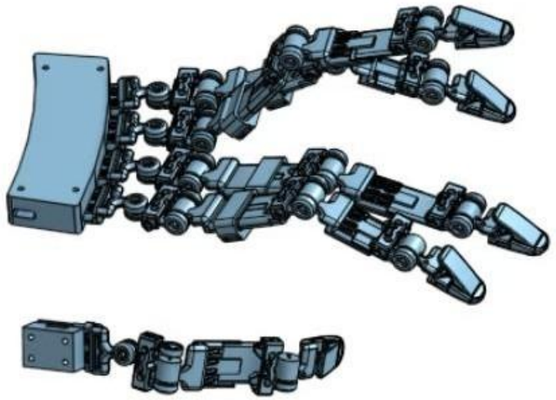
1- Continuité du gant d'acquisition

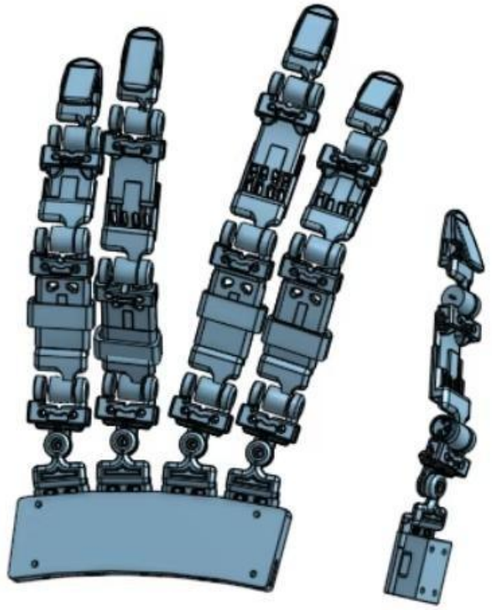
- Réalisation d'un exosquelette muni de potentiomètres avec de déterminer la position de chaque phalanges dans l'espace.

2- Création de la main de la prothèse

- Afin de favoriser la fiabilité et la précision, l'idéal est de limiter l'utilisation de ressorts et d'élastiques. Ici, nous avons privilégié les pignons et les systèmes poulies-courrois.

Assemblage de l'exosquelette :

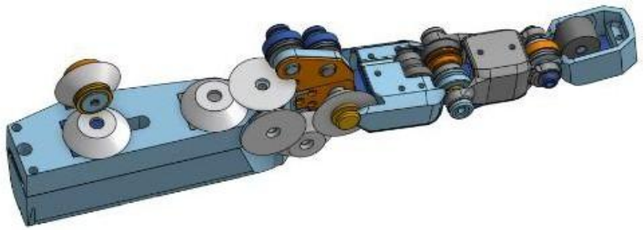


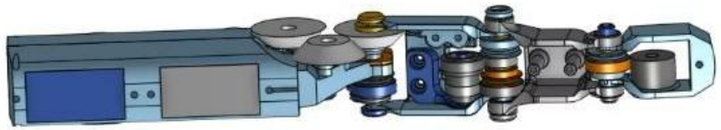


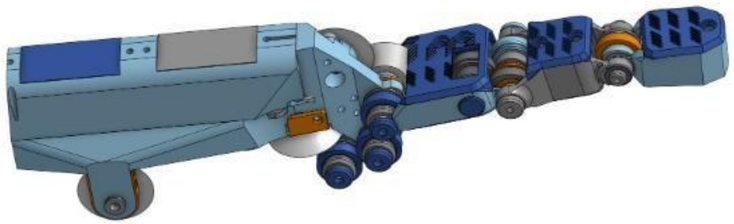


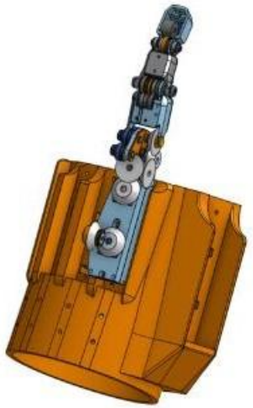
(Capteurs : potentiomètres ajustables 10 kΩ 82MR10K)

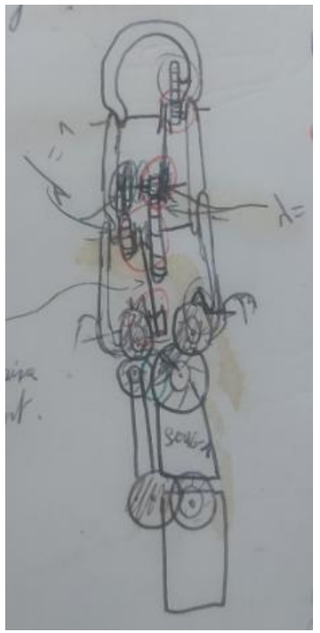
Assemblage de la 'main' de la prothèse :











Pignons pour la Fiabilité

L'utilisation de pignons assure un mouvement robuste et une grande précision.

Systèmes Poulies Courroies

Ces systèmes favorisent une transmission de puissance fluide et efficace.

Vérification que la chaine est bien ouverte

Mobilités = 2, liaisons = 13, pièces = 11, inconnues cinématiques = 20 ;



pour votre attention

Annexe 1 :

Début du programme "carte réceptrice"

```
#include <SPI.h>
#include <Wire.h> // Importing the I2C library.

#define CS_pin 10

float currentTime = 0;
float prevTime = 0;
float deltaTime = 0;


const SPISettings spiSettings(4000000, MSBFIRST, SPI_MODE0);



// MPU6050 I2C addresses
const int8_t I2C_address_MPU1 = 0x68; // AD0 to GND
const int8_t I2C_address_MPU2 = 0x69; // AD0 to VCC (3.3v !)

// Sensor data variables for MPU1 (poignet)
float accel1_x, accel1_y, accel1_z;
float gyro1_x, gyro1_y, gyro1_z;

// Sensor data variables for MPU2 (main)
float accel2_x, accel2_y, accel2_z;
float gyro2_x, gyro2_y, gyro2_z;

//declaration
float velo2_x, velo2_y, velo2_z;
float angvelo2_x, angvelo2_y, angvelo2_z;
float velo1_x, velo1_y, velo1_z;
float angvelo1_x, angvelo1_y, angvelo1_z;

float pos2_x, pos2_y, pos2_z;
float angle2_x, angle2_y, angle2_z;
float pos1_x, pos1_y, pos1_z;
float angle1_x, angle1_y, angle1_z;

// For integration
float accel2_x1=0, accel2_y1=0, accel2_z1=0;
float gyro2_x1=0, gyro2_y1=0, gyro2_z1=0;
float accel1_x1=0, accel1_y1=0, accel1_z1=0;
float gyro1_x1=0, gyro1_y1=0, gyro1_z1=0;

float velo2_x1=0, velo2_y1=0, velo2_z1=0;
float angvelo2_x1=0, angvelo2_y1=0, angvelo2_z1=0;
float velo1_x1=0, velo1_y1=0, velo1_z1=0;
float angvelo1_x1=0, angvelo1_y1=0, angvelo1_z1=0;

float pos2_x1=0, pos2_y1=0, pos2_z1=0;
float angle2_x1=0, angle2_y1=0, angle2_z1=0;
float pos1_x1=0, pos1_y1=0, pos1_z1=0;
float angle1_x1=0, angle1_y1=0, angle1_z1=0;

// Pour Varignon
float L1=0.0; //longueur entre le poignet et la main
float L2=0.0; //longueur entre la main et l'articulaion

//integration
float integration(float A, float a, float deltaTime) {
    return A + a*deltaTime;}

// Preparation des cos
float Cosa;
float Cosb;
float Cosc;
float Sina;
float Sinb;
float Sinc;
```

Annexe 2 : "carte réceptrice" Suite 1

```
float Sinc;

// Composante de la composée
float rotationx(float accel2_x, float accel2_y, float accel2_z) {
    return sin(c) * (accel2_y * sin(α) + accel2_z * cos(α))
+ cos(c) * (
 accel2_x * cos(β)
 - accel2_y * cos(α) * sin(β)
 + accel2_z * sin(α) * sin(β)
);}

float rotationy(float accel2_x, float accel2_y, float accel2_z) {
    return accel2_x*Sinb +(accel2_y*Sina + accel2_z*Cosa)*Cosb;}

float rotationz(float accel2_x, float accel2_y, float accel2_z) {
    return (
- accel2_x * cos(β)
+ accel2_y * sin(α)
+ accel2_y * cos(α) * sin(β)
+ accel2_z * cos(α)
- accel2_z * sin(α) * sin(β)
);}

float vectorielx(float accel2_x, float accel2_y, float accel2_z, float accel1_x, float accel1_y, float accel1_z) {
    return accel2_y*accel1_z - accel2_z*accel1_y;}

float vectoriely(float accel2_x, float accel2_y, float accel2_z, float accel1_x, float accel1_y, float accel1_z) {
    return accel2_z*accel1_x - accel2_x*accel1_y;}

float vectorielz(float accel2_x, float accel2_y, float accel2_z, float accel1_x, float accel1_y, float accel1_z) {
    return accel2_x*accel1_y - accel2_y*accel1_x;}

int16_t raw_accel_x0;  // Raw accelerometer X-axis reading from MPU1 (16-bit signed integer)
int16_t raw_accel_y0;  // Raw accelerometer Y-axis reading from MPU1
int16_t raw_accel_z0;  // Raw accelerometer Z-axis reading from MPU1

int16_t raw_gyro_x0;   // Raw gyroscope X-axis reading from MPU1 (16-bit signed integer)
int16_t raw_gyro_y0;   // Raw gyroscope Y-axis reading from MPU1
int16_t raw_gyro_z0;   // Raw gyroscope Z-axis reading from MPU1

int16_t raw_accel_x;  // Raw accelerometer X-axis reading from MPU2 (16-bit signed integer)
int16_t raw_accel_y;  // Raw accelerometer Y-axis reading from MPU2
int16_t raw_accel_z;  // Raw accelerometer Z-axis reading from MPU2

int16_t raw_gyro_x;   // Raw gyroscope X-axis reading from MPU2 (16-bit signed integer)
int16_t raw_gyro_y;   // Raw gyroscope Y-axis reading from MPU2
int16_t raw_gyro_z;   // Raw gyroscope Z-axis reading from MPU2

void setup() {

  //For integration
  prevTime = millis()/1000.0;



  //I2C 6 axis data receiving



  // Initialisation série pour affichage dans le moniteur série
  Serial.begin(9600);
  Wire.begin();

  // Wake up MPU1
  Wire.beginTransmission(I2C_address_MPU1);
  Wire.write(0x6B); // PWR_MGMT_1 register
```

Annexe 3 : "carte réceptrice" Suite 2

```
  Wire.write(0x6B); // PWR_MGMT_1 register
  Wire.write(0);    // Set to zero (wakes up the MPU-6050)
  Wire.endTransmission(true);

  // Wake up MPU2
  Wire.beginTransmission(I2C_address_MPU2);
  Wire.write(0x6B);
  Wire.write(0);
  Wire.endTransmission(true);

  // Initialisation SPI
  SPI.begin();

  // Déclaration du CS comme sortie
  pinMode(CS_pin, OUTPUT);
  digitalWrite(CS_pin, HIGH);  // Désélectionne l'esclave par défaut

  Serial.println("Master SPI prêt.");
}



// Helper function to convert int16_t into two bytes and send over SPI
void sendInt16(int16_t val) {
  uint8_t bytes[2];
  bytes[0] = val & 0xFF;
  bytes[1] = (val >> 8) & 0xFF;
  SPI.transfer(bytes[0]);
  SPI.transfer(bytes[1]);
}



// Helper function to scale float, convert to int16_t, then send
void sendScaledFloat(float val) {
  int16_t scaled = (int16_t)(val * 100);  // scale factor 100
  sendInt16(scaled);
}



void loop() {

   currentTime = millis()/1000.0;
  deltaTime = currentTime - prevTime;

  // === Read MPU1 ===
  Wire.beginTransmission(I2C_address_MPU1);
  Wire.write(0x3B); // Starting register for Accel data
  byte error1 = Wire.endTransmission(false);  // repeated start, don't release bus

  if (error1 == 0) {
    byte bytesRequested1 = Wire.requestFrom((uint8_t)I2C_address_MPU1, (uint8_t)14, true);
    if (bytesRequested1 == 14 && Wire.available() >= 14) {
      raw_accel_x0 = Wire.read() << 8 | Wire.read();
      raw_accel_y0 = Wire.read() << 8 | Wire.read();
      raw_accel_z0 = Wire.read() << 8 | Wire.read();
      Wire.read(); Wire.read(); // Skip temperature
      raw_gyro_x0 = Wire.read() << 8 | Wire.read();
      raw_gyro_y0 = Wire.read() << 8 | Wire.read();
      raw_gyro_z0 = Wire.read() << 8 | Wire.read();

      accel1_x = (float)raw_accel_x0 / 16384.0 * 9.80665;
      accel1_y = (float)raw_accel_y0 / 16384.0 * 9.80665;
      accel1_z = (float)raw_accel_z0 / 16384.0 * 9.80665;

      gyro1_x = (float)raw_gyro_x0 / 131.0;
```

Annexe 4 : "carte réceptrice" Suite 3

```
      gyro1_x = (float)raw_gyro_x0 / 131.0;
      gyro1_y = (float)raw_gyro_y0 / 131.0;
      gyro1_z = (float)raw_gyro_z0 / 131.0;
    } else {
      Serial.println("Erreur: données MPU1 insuffisantes.");
    }
  } else {
    Serial.print("Erreur I2C MPU1, code: ");
    Serial.println(error1);
  }

  // === Read MPU2 ===
  Wire.beginTransmission(I2C_address_MPU2);
  Wire.write(0x3B);
  byte error2 = Wire.endTransmission(false);

  if (error2 == 0) {
    byte bytesRequested2 = Wire.requestFrom((uint8_t)I2C_address_MPU2, (uint8_t)14, true);
    if (bytesRequested2 == 14 && Wire.available() >= 14) {
      raw_accel_x = Wire.read() << 8 | Wire.read();
      raw_accel_y = Wire.read() << 8 | Wire.read();
      raw_accel_z = Wire.read() << 8 | Wire.read();
      Wire.read(); Wire.read(); // Skip temperature
      raw_gyro_x = Wire.read() << 8 | Wire.read();
      raw_gyro_y = Wire.read() << 8 | Wire.read();
      raw_gyro_z = Wire.read() << 8 | Wire.read();

      accel2_x = (float)raw_accel_x / 16384.0 * 9.80665;
      accel2_y = (float)raw_accel_y / 16384.0 * 9.80665;
      accel2_z = (float)raw_accel_z / 16384.0 * 9.80665;

      gyro2_x = (float)raw_gyro_x / 131.0;
      gyro2_y = (float)raw_gyro_y / 131.0;
      gyro2_z = (float)raw_gyro_z / 131.0;
    } else {
      Serial.println("Erreur: données MPU2 insuffisantes.");
    }
  } else {
    Serial.print("Erreur I2C MPU2, code: ");
    Serial.println(error2);
  }


  //Data treatment



  // For integration

  //EMG value read
  int16_t EMGValue=analogRead(38);

  //Thumb's pot value read
  int16_t Thumb1=analogRead(39);
  int16_t Thumb2=analogRead(40);

  //Index-finger's pot value read
  int16_t Index1=analogRead(41);
  int16_t Index2=analogRead(14);
  int16_t Index3=analogRead(15);

  //Middle-finger's value read
  int16_t Middle1=analogRead(16);
  int16_t Middle2=analogRead(17);
  int16_t Middle3=analogRead(18);

```

Annexe 5 : "carte réceptrice" Suite 4

```
267     velo2_x = integration(velo2_x1, accel2_x, deltaTime);
268     velo1_x = integration(velo1_x1, accel1_x, deltaTime);
269
270     velo2_y = integration(velo2_y1, accel2_y, deltaTime);
271     velo1_y = integration(velo1_y1, accel1_y, deltaTime);
272
273     velo2_z = integration(velo2_z1, accel2_z, deltaTime);
274     velo1_z = integration(velo1_z1, accel1_z, deltaTime);
275
276     angvelo2_x = integration(angvelo2_x1, gyro2_x, deltaTime);
277     angvelo1_x = integration(angvelo1_x1, gyro1_x, deltaTime);
278
279     angvelo2_y = integration(angvelo2_y1, gyro2_y, deltaTime);
280     angvelo1_y = integration(angvelo1_y1, gyro1_y, deltaTime);
281
282     angvelo2_z = integration(angvelo2_z1, gyro2_z, deltaTime);
283     angvelo1_z = integration(angvelo1_z1, gyro1_z, deltaTime);
284
285     pos2_x = integration(pos2_x1, velo2_x, deltaTime);
286     pos1_x = integration(pos1_x1, velo1_x, deltaTime);
287
288     pos2_y = integration(pos2_y1, velo2_y, deltaTime);
289     pos1_y = integration(pos1_y1, velo1_y, deltaTime);
290
291     pos2_z = integration(pos2_z1, velo2_z, deltaTime);
292     pos1_z = integration(pos1_z1, velo1_z, deltaTime);
293
294     angle2_x = integration(angle2_x1, angvelo2_x, deltaTime);
295     angle1_x = integration(angle1_x1, angvelo1_x, deltaTime);
296
297     angle2_y = integration(angle2_y1, angvelo2_y, deltaTime);
298     angle1_y = integration(angle1_y1, angvelo1_y, deltaTime);
299
300     angle2_z = integration(angle2_z1, angvelo2_z, deltaTime);
301     angle1_z = integration(angle1_z1, angvelo1_z, deltaTime);
302
303     // Preparation des cos
304     Cosa=cos(angle2_x - angle1_x);
305     Cosb=cos(angle2_z - angle1_z);
306     Cosc=cos(angle2_y - angle1_y);
307     Sina=sin(angle2_x - angle1_x);
308     Sinb=sin(angle2_z - angle1_z);
309     Sinc=sin(angle2_y - angle1_y);
310
311
312     // pour la main (solide 2)
313     float aox = rotationx(accel2_x, accel2_y, accel2_z);
314     float aoy = rotationy(accel2_x, accel2_y, accel2_z);
315     float aoz = rotationz(accel2_x, accel2_y, accel2_z);
316     float gox = rotationx(gyro2_x, gyro2_y, gyro2_z);
317     float goy = rotationy(gyro2_x, gyro2_y, gyro2_z);
318     float goz = rotationz(gyro2_x, gyro2_y, gyro2_z);
319     float gox1 = rotationx(gyro1_x, gyro1_y, gyro1_z);
320     float goy1 = rotationy(gyro1_x, gyro1_y, gyro1_z);
321     float goz1 = rotationz(gyro1_x, gyro1_y, gyro1_z);
322     float vox = rotationx(velo2_x, velo2_y, velo2_z);
323     float voy = rotationy(velo2_x, velo2_y, velo2_z);
324     float voz = rotationz(velo2_x, velo2_y, velo2_z);
325     float wox = rotationx(angvelo2_x, angvelo2_y, angvelo2_z);
326     float woy = rotationy(angvelo2_x, angvelo2_y, angvelo2_z);
327     float woz = rotationz(angvelo2_x, angvelo2_y, angvelo2_z);
328     float pox = rotationx(pos2_x, pos2_y, pos2_z);
329     float poy = rotationy(pos2_x, pos2_y, pos2_z);
330     float poz = rotationz(pos2_x, pos2_y, pos2_z);
331     float angox = rotationx(angle2_x, angle2_y, angle2_z);
332     float angoy = rotationy(angle2_x, angle2_y, angle2_z);
333     float angoz = rotationz(angle2_x, angle2_y, angle2_z);
```

Annexe 6 : "carte réceptrice" Suite 5

```
343     // preparation de position
344     float pos0x2 = integration(pos2_x1,integration(velo2_x1,aox2, deltaTime), deltaTime);
345     float pos0y2 = integration(pos2_y1,integration(velo2_y1,aoy2, deltaTime), deltaTime);
346     float pos0z2 = integration(pos2_z1,integration(velo2_z1,aoz2, deltaTime), deltaTime);
347     float ang0x2 = integration(angle2_x1,integration(angvelo2_x1,gox, deltaTime), deltaTime);
348     float ang0y2 = integration(angle2_y1,integration(angvelo2_y1,goy, deltaTime), deltaTime);
349     float ang0z2 = integration(angle2_z1,integration(angvelo2_z1,goz, deltaTime), deltaTime);
350     float pos0x1 = integration(pos1_x1,integration(velo1_x1,aox1, deltaTime), deltaTime);
351     float pos0y1 = integration(pos1_y1,integration(velo1_y1,aoy1, deltaTime), deltaTime);
352     float pos0z1 = integration(pos1_z1,integration(velo1_z1,aoz1, deltaTime), deltaTime);
353     float ang0x1 = integration(angle1_x1,integration(angvelo1_x1,gox1, deltaTime), deltaTime);
354     float ang0y1 = integration(angle1_y1,integration(angvelo1_y1,goy1, deltaTime), deltaTime);
355     float ang0z1 = integration(angle1_z1,integration(angvelo1_z1,goz1, deltaTime), deltaTime);
356
357     // Deltas
358     float deltaposx = pos0x2 - pos0x1;
359     float deltaposy = pos0y2 - pos0y1;
360     float deltaposz = pos0z2 - pos0z1;
361     float deltaangx = ang0x2 - ang0x1;
362     float deltaangy = ang0y2 - ang0y1;
363     float deltaangz = ang0z2 - ang0z1;
364
365
366
367     // SPI data transfert to the slave
368
369
370
371     // Démarre la transaction SPI
372     SPI.beginTransaction(spiSettings);
373
374     // Active le CS pour sélectionner l'esclave
375     digitalWrite(CS_pin, LOW);
376
377     // For int16_t values (send directly as two bytes)
378     sendInt16(EMGValue);
379     sendInt16(Thumb1);
380     sendInt16(Thumb2);
381     sendInt16(Index1);
382     sendInt16(Index2);
383     sendInt16(Index3);
384     sendInt16(Middle1);
385     sendInt16(Middle2);
386     sendInt16(Middle3);
387
388     // For float values (scale and send)
389     sendScaledFloat(deltaangx);
390     sendScaledFloat(deltaangy);
391     sendScaledFloat(deltaangz);
392
393     Serial.println(String(deltaposx) + String(deltaposy) + String(deltaposz)); // Should be =0
394
395     // Désactive le CS après communication
396     digitalWrite(CS_pin, HIGH);
397
398     // Termine la transaction
399     SPI.endTransaction();
```

Annexe 7 : "carte réceptrice" Suite 6

```
400
401
402
403     //debugging
404
405
406
407     Serial.println("MPU1:");
408     Serial.print("Accel X: "); Serial.print(accel1_x);
409     Serial.print(", Y: "); Serial.print(accel1_y);
410     Serial.print(", Z: "); Serial.println(accel1_z);
411     Serial.print("Gyro X: "); Serial.print(gyro1_x);
412     Serial.print(", Y: "); Serial.print(gyro1_y);
413     Serial.print(", Z: "); Serial.println(gyro1_z);
414
415     Serial.println("MPU2:");
416     Serial.print("Accel X: "); Serial.print(accel2_x);
417     Serial.print(", Y: "); Serial.print(accel2_y);
418     Serial.print(", Z: "); Serial.println(accel2_z);
419
420
421
422     Serial.print("Gyro X: "); Serial.print(gyro2_x);
423     Serial.print(", Y: "); Serial.print(gyro2_y);
424     Serial.print(", Z: "); Serial.println(gyro2_z);
425
426
427
428   // update for integration
429
430
431
432     prevTime = currentTime;
433
434     accel2_x1 = aox;
435     accel2_y1 = aoy;
436     accel2_z1 = aoz;
437
438     gyro2_x1 = gox;
439     gyro2_y1 = goy;
440     gyro2_z1 = goz;
441
442     accel1_x1 = accel1_x;
443     accel1_y1 = accel1_y;
444     accel1_z1 = accel1_z;
445
446     gyro1_x1 = gyro1_x;
447     gyro1_y1 = gyro1_y;
448     gyro1_z1 = gyro1_z;
449
450     velo2_x1 = vox;
451     velo2_y1 = voy;
452     velo2_z1 = voz;
453
454     angvelo2_x1 = wox;
455     angvelo2_y1 = woy;
456     angvelo2_z1 = woz;
457
458     velo1_x1 = velo1_x;
459     velo1_y1 = velo1_y;
460     velo1_z1 = velo1_z;
461
462     angvelo1_x1 = angvelo1_x;
463     angvelo1_y1 = angvelo1_y;
464     angvelo1_z1 = angvelo1_z;
```

Annexe 8 : "carte réceptrice" Suite 7

```
// update for integration



  prevTime = currentTime;

  accel2_x1 = aox;
  accel2_y1 = aoy;
  accel2_z1 = aoz;

  gyro2_x1 = gox;
  gyro2_y1 = goy;
  gyro2_z1 = goz;

  accel1_x1 = accel1_x;
  accel1_y1 = accel1_y;
  accel1_z1 = accel1_z;

  gyro1_x1 = gyro1_x;
  gyro1_y1 = gyro1_y;
  gyro1_z1 = gyro1_z;

  velo2_x1 = vox;
  velo2_y1 = voy;
  velo2_z1 = voz;

  angvelo2_x1 = wox;
  angvelo2_y1 = woy;
  angvelo2_z1 = woz;

  velo1_x1 = velo1_x;
  velo1_y1 = velo1_y;
  velo1_z1 = velo1_z;

  angvelo1_x1 = angvelo1_x;
  angvelo1_y1 = angvelo1_y;
  angvelo1_z1 = angvelo1_z;

  pos2_x1 = pox;
  pos2_y1 = poy;
  pos2_z1 = poz;

  angle2_x1 = angox;
  angle2_y1 = angoy;
  angle2_z1 = angoz;

  pos1_x1 = pos1_x;
  pos1_y1 = pos1_y;
  pos1_z1 = pos1_z;

  angle1_x1 = angle1_x;
  angle1_y1 = angle1_y;
  angle1_z1 = angle1_z;

}
```

Annexe 9 : "asservissement moteur"

```
from py2duino import *
import time
import matplotlib.pyplot as plt
import numpy as np

ar1 = Arduino(3) ## Le port <<COM>> sur lequel est branchée l'arduino


pinA = 2   ## Le port D2 est associé à l'interruption 0
pinB = 3


dir1 = 2
dir2 = 3
pwmPin = 9

EMGpin=A0
EMGValue=analogRead(EMGpin)

ar1.pinMode(4,'OUTPUT')
ar1.pinMode(5,'OUTPUT')

codeur=Encoder(ar1,pinA,pinB,4)

def integrale(u,t1,t0):
    return((u(t1)-u(t0))*(t1-t0)/2)

def PI (epsilon, K, Ti, t1, t0):
    return(K*epsilon+integrale(u, t1, t0)/Ti)


def ControlTest (consigne, durée, N):
    "Motor to EMG link test"
    assert(0<=consigne<=255)

    K=0
    pos,i=0,0       #position initialisation
    a=1             #réglage du sens de la rotation
    b=1-a
    temps=[]
    deg=[]
    position=[0]
    t0=time.perf_counter()
    t1=time.perf_counter()
    t=t1-t0
    temps.append(t)

    ar1.digitalWrite(dir1,'HIGH')       #direction initialisation
    ar1.digitalWrite(dir2,'LOW')

    while t<durée :
        epsilon=consigne-codeur.read()*4*7*2*np.pi/(37*50)       #comparator
        commande=min(abs(PI(K,t,epsilon), 255))    #correction

        if epsilon>0:       #direction
            ar1.digitalWrite(dir1,a)
            ar1.digitalWrite(dir2,b)
        else:
            ar1.digitalWrite(dir1,b)
            ar1.digitalWrite(dir2,a)

        ar1.analogWrite(pwmPin,commande)    #envoi de la commande au moteur

        i+=1                #updating lists
        T1=time.perf_counter()
        t=t1-t0
        temps.append(t)
        position.append(pos)
        deg.append(coeur())


    ar1.analogWrite(pwmPin,0)      #reset position
    print(len(temps,deg))
    print("test terminé")

    plt.plot(temps[:1],deg)     #drawing curve
```

Annexe 10 : "carte relay"

```ino
#include <ESP32SPISlave.h>

// SPI Slave setup
ESP32SPISlave SPI_Slave;

#define PIN_MISO  12
#define PIN_MOSI  11
#define PIN_SCLK  10
#define PIN_CS     9

// Data buffers
uint8_t recvBuffer[30];  // 15 int16_t values = 30 bytes
uint8_t sendBuffer[1];   // not used for now

void setup() {
  Serial.begin(115200);
  Serial.println("ESP32 SPI Slave + multiple serials ready.");

  // SPI Slave configuration
  SPI_Slave.setDataMode(SPI_MODE0);
  SPI_Slave.begin(HSPI, PIN_SCLK, PIN_CS, PIN_MISO, PIN_MOSI);

  // Other serial ports
  Serial1.begin(115200, SERIAL_8N1, 4, 2);  // RX=4, TX=2
  Serial2.begin(115200, SERIAL_8N1, 14, 13); // RX=14, TX=13

  Serial.println("All serial ports initialized.");
}

void loop() {
  // Wait for SPI transfer
  size_t receivedLength = SPI_Slave.transfer(
    sendBuffer,
    recvBuffer,
    sizeof(recvBuffer),
    1000  // 1s timeout
  );

  if (receivedLength >= sizeof(recvBuffer)) {
    int16_t receivedData[15];
    // convert bytes to int16_t
    for (int i = 0; i < 15; i++) {
      receivedData[i] = recvBuffer[2 * i] | (recvBuffer[2 * i + 1] << 8);
    }

    // Process values
    int16_t EMGValue = receivedData[0];
    int16_t Thumb1 = receivedData[1];
    int16_t Thumb2 = receivedData[2];
    int16_t Index1 = receivedData[3];
    int16_t Index2 = receivedData[4];
    int16_t Index3 = receivedData[5];
    int16_t Middle1 = receivedData[6];
    int16_t Middle2 = receivedData[7];
    int16_t Middle3 = receivedData[8];

    float angx = receivedData[9] / 100.0;
    float angy = receivedData[10] / 100.0;
    float angz = receivedData[11] / 100.0;

    int16_t Index02 = (Index2 * Index3) / 2;
    int16_t Middle02 = (Middle2 * Middle3) / 2;

```

Annexe 11 : "carte relay" Suite

```
  SPI_Slave.setDataMode(SPI_MODE0);
  SPI_Slave.begin(HSPI, PIN_SCLK, PIN_CS, PIN_MISO, PIN_MOSI);

  // Other serial ports
  Serial1.begin(115200, SERIAL_8N1, 4, 2);  // RX=4, TX=2
  Serial2.begin(115200, SERIAL_8N1, 14, 13); // RX=14, TX=13

  Serial.println("All serial ports initialized.");
}

void loop() {
  // Wait for SPI transfer
  size_t receivedLength = SPI_Slave.transfer(
    sendBuffer,
    recvBuffer,
    sizeof(recvBuffer),
    1000  // 1s timeout
  );

  if (receivedLength >= sizeof(recvBuffer)) {
    int16_t receivedData[15];
    // convert bytes to int16_t
    for (int i = 0; i < 15; i++) {
      receivedData[i] = recvBuffer[2 * i] | (recvBuffer[2 * i + 1] << 8);
    }

    // Process values
    int16_t EMGValue = receivedData[0];
    int16_t Thumb1 = receivedData[1];
    int16_t Thumb2 = receivedData[2];
    int16_t Index1 = receivedData[3];
    int16_t Index2 = receivedData[4];
    int16_t Index3 = receivedData[5];
    int16_t Middle1 = receivedData[6];
    int16_t Middle2 = receivedData[7];
    int16_t Middle3 = receivedData[8];

    float angx = receivedData[9] / 100.0;
    float angy = receivedData[10] / 100.0;
    float angz = receivedData[11] / 100.0;

    int16_t Index02 = (Index2 * Index3) / 2;
    int16_t Middle02 = (Middle2 * Middle3) / 2;

    // Debug over USB Serial
    Serial.println("--- Données reçues ---");
    Serial.print("EMGValue: "); Serial.println(EMGValue);
    Serial.print("Thumb1: "); Serial.println(Thumb1);
    Serial.print("Thumb2: "); Serial.println(Thumb2);
    Serial.print("Index1: "); Serial.println(Index1);
    Serial.print("Index2: "); Serial.println(Index2);
    Serial.print("Index3: "); Serial.println(Index3);
    Serial.print("Middle1: "); Serial.println(Middle1);
    Serial.print("Middle2: "); Serial.println(Middle2);
    Serial.print("Middle3: "); Serial.println(Middle3);
    Serial.print("angx: "); Serial.println(angx, 4);
    Serial.print("angy: "); Serial.println(angy, 4);
    Serial.print("angz: "); Serial.println(angz, 4);
    Serial.println("________ end of cycle ________");

    // Send over other serials
    Serial1.println(String(angx, 2) + "," + String(EMGValue);
    Serial2.println(String(angy, 2) + "," + String(angz, 2) + "," + String(Thumb1) + "," + String(Thumb2) + "," + String(Index1) + "," + String(Index02) + "," + String(Middle1) + "," + String(Middle02));
  }

  delay(1);
}
```

Annexe 12 : "carte servos"

```ino
#include <ESP32_Servo.h>

constexpr int NUM_SERVOS = 11;
Servo servos[NUM_SERVOS];
const int servoPins[NUM_SERVOS] = {2, 4, 5, 13, 14, 15, 16, 17, 18, 19, 21};

constexpr unsigned long READ_TIMEOUT_MS = 1000;
constexpr int MAX_LINE_LENGTH = 100;  // max expected chars for 11 angles + commas

// Buffer to store incoming line
char lineBuffer[MAX_LINE_LENGTH + 1] = {0};

// Reads a line from Serial2 with timeout (non-blocking)
bool readLineFromSerial2(char *buffer, size_t maxLen, unsigned long timeout = READ_TIMEOUT_MS) {
  unsigned long start = millis();
  size_t pos = 0;

  while (millis() - start < timeout && pos < maxLen - 1) {
    while (Serial2.available()) {
      char c = Serial2.read();
      if (c == '\n') {
        buffer[pos] = '\0';
        return true;
      }
      if (c != '\r') {  // ignore carriage return
        buffer[pos++] = c;
      }
    }
  }
  buffer[pos] = '\0';  // null terminate even on timeout
  return (pos > 0);
}

// Parse comma-separated angles from a char array into int array
bool parseAngles(const char *line, int *angles, int count) {
  int index = 0;
  const char *start = line;

  while (index < count) {
    const char *comma = strchr(start, ',');
    if (comma == nullptr) {
      if (index == count - 1) {
        // Last angle, no comma expected
        angles[index] = atoi(start);
        index++;
        break;
      } else {
        return false;  // not enough commas
      }
    } else {
      // Extract substring for current angle
      int length = comma - start;
      if (length <= 0) return false;  // empty angle
      char temp[6];  // enough for angle string like "180\0"
      if (length >= sizeof(temp)) return false; // too long angle
      strncpy(temp, start, length);
      temp[length] = '\0';
      angles[index] = atoi(temp);
      index++;
      start = comma + 1;
    }
  }
  return (index == count);
}

// Smoothly move servos to target angles over given steps
void moveServosSmoothly(int *targetAngles, int steps = 20, int delayMs = 20) {
  static int currentAngles[NUM_SERVOS] = {90, 90, 90, 90, 90, 90, 90, 90, 90, 90, 90};
```

Annexe 13 : "carte servos" Suite

```
// Smoothly move servos to target angles over given steps
void moveServosSmoothly(int *targetAngles, int steps = 20, int delayMs = 20) {
  static int currentAngles[NUM_SERVOS] = {90, 90, 90, 90, 90, 90, 90, 90, 90, 90, 90};

  for (int step = 1; step <= steps; step++) {
    for (int i = 0; i < NUM_SERVOS; i++) {
      int angle = currentAngles[i] + (targetAngles[i] - currentAngles[i]) * step / steps;
      angle = constrain(angle, 0, 180);
      servos[i].write(angle);
    }
    delay(delayMs);
  }

  // Update currentAngles for next move
  for (int i = 0; i < NUM_SERVOS; i++) {
    currentAngles[i] = constrain(targetAngles[i], 0, 180);
  }
}

void setup() {
  Serial.begin(115200);
  Serial2.begin(115200);  // UART master connected here
  Serial.println("Initializing 11 servos...");

  for (int i = 0; i < NUM_SERVOS; i++) {
    servos[i].attach(servoPins[i], 500, 2400);
    servos[i].write(90); // initialize at middle position
    Serial.printf("Servo %d attached to pin %d\n", i, servoPins[i]);
  }

}

void loop() {
  if (Serial2.available()) {
    if (readLineFromSerial2(lineBuffer, sizeof(lineBuffer))) {
      // Remove trailing whitespace
      int len = strlen(lineBuffer);
      while (len > 0 && isspace(lineBuffer[len - 1])) {
        lineBuffer[--len] = '\0';
      }

      int angles[NUM_SERVOS] = {0};
      if (parseAngles(lineBuffer, angles, NUM_SERVOS)) {
        // Constrain all angles and move smoothly
        for (int i = 0; i < NUM_SERVOS; i++) {
          angles[i] = constrain(angles[i], 0, 180);
        }
        moveServosSmoothly(angles);

        Serial.print("Servos updated: ");
        for (int i = 0; i < NUM_SERVOS; i++) {
          Serial.print(angles[i]);
          if (i < NUM_SERVOS - 1) Serial.print(", ");
        }
        Serial.println();

        Serial2.println("OK"); // Acknowledge success
      } else {
        Serial.print("Invalid data received: ");
        Serial.println(lineBuffer);
        Serial2.println("ERR"); // Send error back
      }
    }
  }
}
```